Evolving Systems (2025) 16:86
https://doi.org/10.1007/s12530-025-09714-9

ORIGINAL PAPER

# On resolving the out of vocabulary problem in DisCoCat-based quantum natural language processing

**Amey Bhatuse[1] · Ankit Khandelwal[2] · Sandeep S. Udmale[1] · M Girish Chandra[2] · Sopan Kolte[1]**

Received: 13 February 2025 / Accepted: 18 June 2025 / Published online: 8 July 2025
© The Author(s), under exclusive licence to Springer-Verlag GmbH Germany, part of Springer Nature 2025

**Abstract**
The field of quantum computing has evolved a lot in the past few years with doors opening for novel applications in a variety of fields. Consequently, natural language processing (NLP) has also been affected due to this disruption, leading to various proposals for doing NLP on a quantum computer. One such promising framework is DisCoCat, a linguistics-oriented model designed to capture the meaning of language without compromising its structural information. Recently, DisCoCat models have been run on real quantum hardware. However, a few issues in the current formulation have led to problems like the out-of-vocabulary (OOV) problem. In this work, our aim is to tackle this OOV problem with respect to DisCoCat-based quantum natural language processing. We first design a novel functor mapping from string diagrams, a primary construction in DisCoCat, to quantum circuits. Later, we test the correctness and validity of this new technique on state-of-the-art datasets for tasks such as sentence similarity and paraphrase identification. We present clear empirical evidence that this novel functor mapping not only performs better than the pre-existing technique in most cases, but also it does so with relatively fewer parameters, thus ensuring better scalability with improved performance.

**Keywords** DisCoCat · Discrete Circuit Ansatz (DCA) · Natural language processing (NLP) · Out of vocabulary (OOV) · Quantum computing

## 1 Introduction

The advent of understanding natural languages has witnessed tremendous progress in the past few years with the recent rise of technologies such as large language models (LLMs) (Dong et al. 2019). Briefly put, these LLMs are highly advanced machine learning systems capable of extracting valuable information from massive datasets that can be used for downstream applications like text generation or translation. The huge success of these models can be attributed to the fact that they have multimodal capabilities, i.e. the ability to handle multiple types of data such as text, audio, images, or video. However, two core requirements for these models to perform better are the availability of increasingly large datasets and an extensive computing infrastructure to learn model parameters from this huge data. Thus, finding alternate ways of processing natural language with manageable resource requirements can be considered an important research direction. On the same note, quantum computing is an emerging field based on a theoretical structure built using the inherent randomness of quantum physical systems. If certain conditions are met, quantum computers give an exponential complexity separation against classical computation (Shor 1997). A recent survey (Guarasci et al. 2022) on the use of quantum computers for NLP applications mentions various approaches to obtain a quantum advantage, that is, when a quantum computer can compute a task that classical computers cannot or can do with less precision or more

✉ Sandeep S. Udmale
ssudmale@it.vjti.ac.in

Amey Bhatuse
asbhatuse_b20@ce.vjti.ac.in

Ankit Khandelwal
khandelwal.ankit3@tcs.com

M Girish Chandra
m.gchandra@tcs.com

Sopan Kolte
sgkolte@ce.vjti.ac.in

[1] Department of Computer Engineering and Information Technology, Veermata Jijabai Technological Institute (VJTI), Mumbai, Maharashtra 400019, India

[2] Tata Consultancy Services (TCS), Bengaluru, Karnataka, India

energy. In summary, this quest is motivated by the following reasons:

- The inherent properties of quantum systems, such as quantum superposition and state evolution, have some correspondence to the NLP concepts of language ambiguity and evolution, respectively (Wu et al. 2021).
- The state-of-the-art NLP methods have large resource requirements (Hadi et al. 2023), which can be reduced by leveraging the non-trivial computational complexity improvements of quantum computers over classical computers.

Some works employing hybrid classical and quantum techniques involve aspects such as replacing a part of existing NLP models with quantum neural networks (Tomal et al. 2025; Smaldone et al. 2025). The linguistics field has witnessed some pioneering contributions in the past two decades. For example, the DisCoCat (Coecke et al. 2010) framework proposed by Bob Coecke et al. encodes natural language text into abstract entities called string diagrams followed by their transformation into mathematical objects like tensor networks or quantum circuits to infer meaning from that text. This framework has gained widespread popularity with the simultaneous successful advent of quantum machine learning (Huang et al. 2021), a dawning field at the intersection of quantum computing and machine learning. There has been some progress in using QML in conjunction with DisCoCat, for example, in resolving pronouns (Wazni et al. 2024) or even working with multiple modes of data such as images and text (Hawashin and Sadrzadeh 2024). Researchers have also tried to build NLP models inspired by quantum-theoretical processes (Wang et al. 2021; Zhu et al. 2023; Huang et al. 2024). However, such models only draw on relevant concepts from quantum theory rather than being actually capable of running on a real quantum computer.

In this paper, we propose a new way of mapping string diagrams to quantum circuits called the Discrete Circuit Ansatz [1] (DCA) to tackle the out-of-vocabulary (OOV) problem with respect to DisCoCat. As the name suggests, an OOV problem arises when words that do not occur in the training set for a model are found during inference on unseen data. This problem has also arisen in the DisCoCat way of doing NLP, while creating quantum circuits using the information of the word and its grammar type. Although it has plagued the classical NLP domain for a long time, its ramifications become much more pronounced in DisCoCat once the number of such words increases in a dataset. This is due to the techniques currently employed to infer meaning from text using DisCoCat. A very recent work (Rubio-Perez 2024) on the topic tries to solve the same problem using a hybrid combination of quantum and classical methods. Although the core intuition of their solution seems to be similar to the one used in our work, we provide a more elaborate study of the problem by introducing more novel ideas and testing our approach against state-of-the-art datasets. Motivated by this fact, we propose a simple effective idea to mitigate this issue. As we shall see further, based on this idea, we employ a novel method called DCA which is not only more scalable but also provides better results than the pre-existing mapping. The paper is structured as follows: Sect. 2 provides the background necessary to understand the contribution of this paper. In particular, Sects. 2.1 and 2.2 explain the foundational theoretical details of DisCoCat and its connection to quantum computing. Section 2.3 lists the main steps in the pipeline for the practical application of DisCoCat to perform quantum natural language processing (QNLP). Section 2.4 explains how the OOV problem appears in DisCoCat-based QNLP. Section 3 describes the main contribution of this paper. Sections 4 provide the specific empirical formulation of DCA used to obtain the results with some key limitations presented in this paper. Section 5 concludes the paper with possible future directions.

## 2 Background

In this section, we present some foundational concepts required to understand the problem mentioned in Sect. 2.4 and the solution methodology proposed in Sect. 3.1

### 2.1 DisCoCat

DisCoCat is a unique syntax-based model that incorporates the grammatical structure of text via formal rules (Steedman and Baldridge 2011) in conjunction with its distributional information (Harris 1954) to give meaning to the text. It first builds different kinds of *types* for grammatical units in the text, such as $v$ for verbs, $adj$ for adjectives, $p$ for phrases, and $s$ for sentences, in a structure called a *category*, and then models the meanings of these types by a map, which additionally preserves the internal structure between the various types of that category. An object that collates the separate words along with their grammar types in an intuitive visual representation based on the whole sentence is called a string diagram. A detailed discussion on the entire formulation of DisCoCat is beyond the scope of this paper. Interested readers may refer to Coecke et al. (2010). In short, DisCoCat capitalizes on the similarity between the underlying structure of Lambek's pregroup grammar (Lambek 1997) and the category of finite-dimensional vector spaces. It first derives a pregroup-grammatical construction for the text and then maps each grammar type to corresponding vector spaces

[1] A circuit Ansatz here refers to a quantum algorithm subroutine used to create specific kinds of quantum circuits.

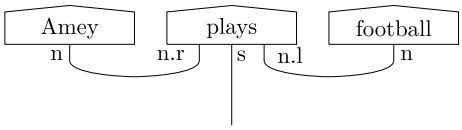



**Fig. 1** DisCoCat diagram for *Amey plays football*

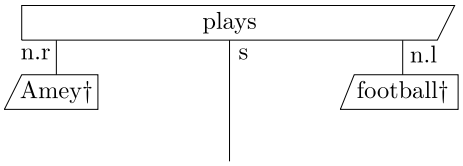



**Fig. 2** Rewritten DisCoCat diagram for *Amey plays football*

for semantics. The meaning of the whole text can then be derived depending on the compositional rules for the predefined grammar types as the internal structure between them is not altered in the mapping. This possibility of mapping words and their abstract types to their quantitative meaning without altering the structure of connections between those words is what makes DisCoCat a highly efficient framework for modeling the true meaning of text.

For example, Fig. 1 shows the string diagram for the sentence—*Amey plays football*. The pregroup grammar types for the words are $n$ for Amey and football, and $n.r\ s\ n.l$ for play. The type $n$ denoting a noun is called an *atomic* type, and $n.l$ and $n.r$ can be considered as *derived* types or *adjoints* of $n$ satisfying certain pregroup grammar reduction rules. The word *play* is connected to both *Amey* and *football* and hence assigned a *composite* type $n.r\ s\ n.l$, created from atomic type $s$ and adjoints of type $n$. The *wires* connecting the words diagrammatically denote the reduction rules. The free wire of atomic type $s$ is assumed to hold the meaning of the entire sentence.

## 2.2 Quantum DisCoCat

Though there are various models of quantum computation, we are only concerned with the quantum circuit-based model, also called gate-based quantum computation because string diagrams have a natural mapping to quantum circuits. Reference Bhat et al. (2022) provides a comprehensive description of the basics of practical quantum computing. Some fundamental concepts necessary to understand our contribution are listed below.

**Qubits**: A qubit is a basic unit of information in quantum computing. The state of a qubit is represented using a unit vector in a 2-dimensional complex vector space ($C^2$). It can be in two distinct states of $|0\rangle$ or a $|1\rangle$, similar to a bit, its classical counterpart, or be a superposition $|\psi\rangle$ of these distinct states.

**Quantum gates**: These are unitary matrices that act on the complex vector space of qubits.

**Quantum circuit**: A quantum circuit is a collection of qubits acted upon by quantum gates to get a desired output.

**Measurements**: A measurement operation projects a qubit state vector to a distinct quantum state, information of which can later be used for classical post-processing.

In Meichanetzidis et al. (2021), the diagrammatic formalism of DisCoCat has been reformulated in terms of quantum theory, where the wires denote qubits, the pentagons represent initial quantum states for the attached qubits, and the connections between different qubits denote some quantum gates involving those qubits. It suggests that the evolution of this whole quantum system and the eventual measurement give the meaning of the whole sentence. DisCoCat can also be considered to be highly compatible with quantum systems due to the use of vector spaces to describe quantitative aspects of each of them. This may imply that a quantum computer can handle natural language in a highly efficient way; however, rigorous theory and its subsequent experimental validation would be needed to warrant this possibility. Now that we have understood the fundamentals, let us see how these tools are used in practice to do NLP.

## 2.3 DisCoCat-based QNLP

Reference Lorenz et al. (2023) proposed a general pipeline for performing NLP tasks using the quantum-theoretical formulation of DisCoCat. Each part of the pipeline is briefly mentioned below.

**String diagram generation:** In the first step, a sentence is parsed using a statistical parser resulting in a string diagram. This string diagram is created on the basis of the pregroup grammar type assigned to each word by the parser. An example of a string diagram is shown in Fig. 1.

**Rewriting string diagrams:** For larger text, these diagrammatic derivations of text in grammar form can become significantly complex, which may lead to inordinate use of hardware resources and exceedingly long training times. To tackle this, the string diagrams are rewritten into a simpler representation using some native category theory operations. Figure 2 shows a rewritten diagram for the diagram shown in Fig. 1.

**Creating quantum circuits:** Once rewritten, a *functor*[2] maps these diagrams to parameterized quantum circuits (PQCs). A PQC can be considered as a function that takes as input an initial quantum state and a few trainable parameters,

[2] In category theory, a map between categories is called a functor. Here, there are two categories, namely string diagrams and quantum circuits.

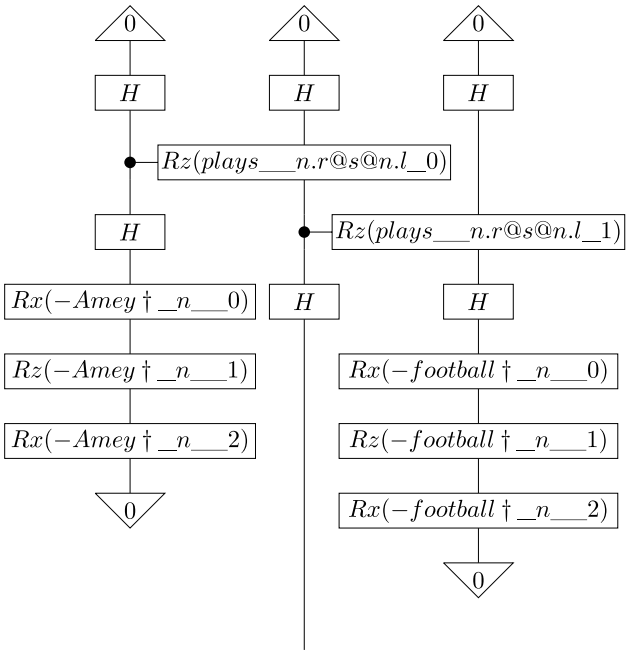



**Fig. 3** IQP circuit for *Amey plays football*

and outputs a resulting quantum state due to application of gates incorporating those trainable parameters on the initial quantum state. The circuit is constructed in a top-down manner, assuming the phenomenon of wires coming out from the bottom of a box in a string diagram as the initialization of qubits. The triangle with 0 written inside it, at the top of each qubit, indicates the initialization of that qubit with a $|0\rangle$ state. Each wire in the string diagram is mapped to a certain number of qubits which can be pre-decided for each atomic type. Figure 3 shows a quantum circuit obtained from the rewritten diagram in Fig. 2. In that, the atomic types $n$ and $s$ are assigned one qubit each, the composite type $n.r\, s\, n.l$ eventually gets three qubits, one from type $s$ and two from adjoints $n.r$ and $n.l$ of type $n$ for words *Amey* and *football* respectively. Single-qubit parameteric gates replace atomic types, whereas composite types are replaced by multi-qubit gates. Figure 3 is an instantaneous quantum polynomial (IQP) (Shepherd and Bremner 2009) circuit with Hadamard gate (H), parameterized *RX* and *RZ* single-qubit gates, and parameterized *CRZ* gate for multi-qubit operations. Each of these operations can be represented by unique unitary matrices.

Using this procedure of assigning qubits to grammar types in case of the string diagram in Fig. 1, it is not difficult to see that a quantum circuit obtained from it would have had two more qubits than the circuit in Fig. 3. Thus, in the context of QNLP, the rewriting procedure helps reduce the qubit count. In addition, this consequently eases a process called postselection. Postselection refers to the process of conditioning the outcome of a measurement of a qubit on a certain predefined output for some other qubit. In Fig. 3, the inverted triangles with 0 written inside, at the bottom of each qubit, indicate postselecting the output of the circuit given that those qubits are in the state $|0\rangle$. It is important to understand that we cannot predefine the output of a qubit in a real sense. So, for experimental procedures, we run the same circuit multiple times and then filter out those outputs which do not satisfy the postselection condition. The more qubits, the more circuit runs will be required to get a better estimate of the output and hence more training time. So, to ensure a circuit with less qubits for a diagram, it is rewritten. Having understood the DisCoCat way of doing NLP, we can now look at the actual problem in hand.

### 2.4 Problem statement

As mentioned earlier, OOV words pose a significant barrier to model generalization. Few works have been proposed to address this problem in the classical NLP domain (Pinter et al. 2017; Garneau et al. 2018). In the DisCoCat framework, the OOV problem arises after creating a PQC. As is visible from Fig. 3, the parameters for the circuit are created based on both the word and its type. The symbols for parameters of the quantum circuit are -

*Amey* † *_n__*0 , *Amey* † *_n__*1 , *Amey* † *_n__*2
*plays__n.r@s@n.l_*0, *plays__n.r@s@n.l_*1
*football* † *_n__*0, *football* † *_n__*1, *football* † *_n__*2

Each of the parameters can be randomly initialized and then trained depending on the task at hand. However, this way of parameterization poses a huge problem while inference, because if an unknown word is encountered, its corresponding parameters would not be a part of the training process due to the need of the actual word string to form the parameter symbol. Thus, this will lead to an analogous situation in classical NLP models where usually random embeddings or scarcely learned morphology-based embeddings (Sasaki et al. 2019) are assigned to unknown words.

## 3 Methodology

In this section, we explain in detail the method used to deal with the OOV problem. We term this novel method Discrete Circuit Ansatz (DCA), which is essentially a different way of mapping string diagrams to quantum circuits. First, the general DCA formulation is explained in Sect. 3.1. Later, Sect. 3.2 presents a few ways to create working models using DCA, which can also be used as inspiration to derive more complex and sophisticated DCA-based models. Section 3.3 lists the various NLP tasks performed using DCA-based models. The datasets used in testing of DCA-based models along with the preprocessing routines are mentioned in Sect. 3.4.

### 3.1 Discrete Circuit Ansatz

Here, we propose a novel functor called Discrete Circuit Ansatz (DCA) to tackle this issue by separating the word and it's type, and use information gained from each in a discrete sense. The DCA parameterizes only the type of words. The word itself is assigned a unique quantum state which is loaded onto the quantum circuit before applying the type-specific parametric gates to the circuit. The unique quantum state for each word is assumed to be known before applying DCA. Thus, for an unknown word, the only thing required to be modeled is the type of the word which decides the context of that word in the given text, since we already have a representation for the word itself. An important thing to note before moving forward is that, even though the core idea of DCA aims to solve the OOV problem, the exact formulation of the idea used in this work does not seem to do it in its entirety. Rather, what we achieve is a way to make the solution to the problem less remote without compromising the rigorous mathematical structure of DisCoCat built to model natural language. This emphasizes the fact that the major contribution of this paper is an empirical validation of a novel way of solving the OOV problem, whereas a rigorous theory to support the claim is currently due. A discussion regarding this can be found in Sect. 5.

Since we do not wish to include the word information in the learning routine, there is a strong need for a sufficiently accurate way to represent this part. This is because a suboptimal representation for the word part will eventually hinder performance or may even slow down progress by acting as a bottleneck. We try to leverage the extensive research in the classical NLP domain to get this part right. The FastText model (Bojanowski et al. 2017) based on the Skip Gram technique is known to learn word embeddings with high efficiency. In this work, we employ pre-trained embeddings for words from a FastText model trained on the CommonCrawl and Wikipedia corpora (Grave et al. 2018). These word embeddings are later passed to DCA to create the quantum state for each word using the Mottonen state preparation routine (Möttönen et al. 2005). This routine enables us to create an arbitrary quantum state on a given set of qubits using uniformly controlled rotations.

**Algorithm 1** Discrete Circuit Ansatz functor

$\tau$ = Diagram box, $W$ = word vector dictionary, $T$ = Dictionary having atomic types as keys and corresponding qubit count as values
Create dictionary $D$ with keys *word* and *type*, *word* stores character string of the word and *type* stores type of the word
$wv \leftarrow W[D[word]]$ {Retrieve FastText-based vector representation for the word}
**if** $D[type]$ is atomic **then**
    $s \leftarrow T[type]$
**else**
    $l \leftarrow len(D[type])$ {Find total wires used by the compound type}
    $s \leftarrow l * T[type]$ {Total qubit count scales linearly with wire count of compound type}
**end if**
$quantum_state_vector \leftarrow wv[0 : 2^s]$ {Truncate $wv$ keeping only first $2^s$ elements}
$C \leftarrow MottonenStatePreparation(quantum_state_vector)$ {$n$-qubit state requires $2^s$ size vector}
Append type-specific PQC to $C$

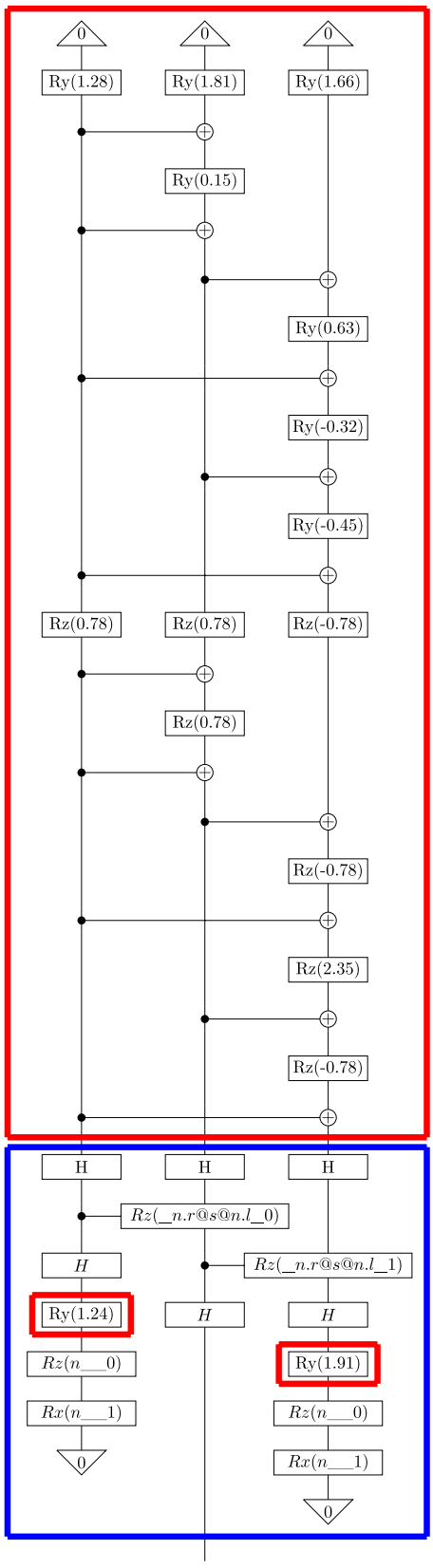



**Fig. 4** Example of a DCA circuit. Red boxes indicate non-parameterized parts of the circuit corresponding to word information. The blue box indicates parameterized gates corresponding to grammar type of words

The Algorithm 1 describes the procedure for converting a box in a string diagram to its corresponding quantum circuit representation. To encode the word embeddings onto the quantum circuit, the Mottonen state preparation routine is employed. This routine assumes the input vector to be a quantum state representation for the word and tries to encode this state onto the designated qubits using rotation and controlled NOT gates. In Fig. 4, the parts of the quantum circuit outlined by the red boxes are obtained using the Mottonen routine. As is visible, the higher the number of qubits $n$, larger is the input vector of size $2^n$, which consequently increases the depth of the circuit. Since the input size is dependent on the qubit count of the words, the embeddings need to be accordingly truncated, an operation which turns out to be inherently lossy. To mitigate this information loss, PCA (principal component analysis) (Maćkiewicz and Ratajczak 1993) is first applied to the pre-trained embeddings before being passed to DCA. This is to ensure that the truncation step retains maximum information of the data by including the most significant components for quantum state creation. Rather than manually setting the maximum explained variance threshold which would require a separate investigation for its appropriate value, we instead used the Minka MLE (Minka 2000) approach, which is already implemented in the Scikit Learn library (Pedregosa et al. 2018) to adapt the number of principal components depending on the underlying embeddings. The exact PCA model used in our experiments is mentioned in Sect. 3.2.

After applying the DCA ansatz, the quantum circuit (Fig. 4) for *Amey plays football* would have trainable parameters that look like the following:

*n*__0, *n*__1, *n*__2
*_n.r@s@n.l_0*, *_n.r@s@n.l_1*

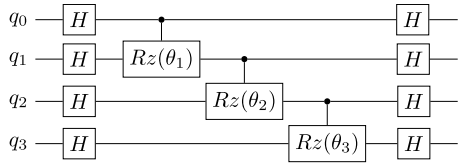



**Fig. 5** Schematic figure of an IQP circuit

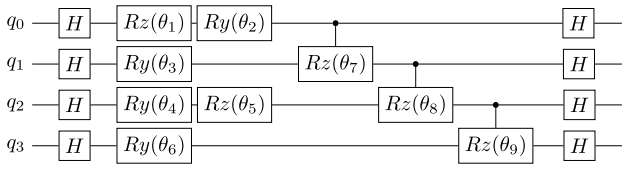



**Fig. 6** Schematic figure of a DCA—Type 1 circuit for a box of type *n.l s n.r n*

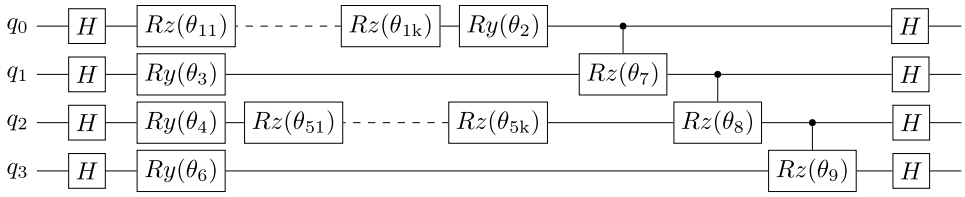



**Fig. 7** Schematic figure of a DCA—Type 2 circuit for a box of type $n.l_1 \dots l_k\ s\ n.r_1 \dots r_k\ n$

Here, it is evident that the word string itself is not included in the parameter symbols. The insight behind the word and type separation phenomenon arose from the fact that for every newly added word, its associated grammar type is relatively more probable to be in training set. With respect to the grammar types, consider that we have a large enough training dataset and sufficient computational resources, we can, upto a certain extent, hypothesize that the variety of types will be limited enough to be captured by these training data. Due to this, the grammar type for any new word appearing in the test set will have a high probability of presence in the train set. So, if we are confident to have a good enough quantum state representation for the newly added word and the type of the word already present in the train set, then, in principle, DCA serves as a fairly good candidate in solving the OOV problem.

### 3.2 DCA-based QNLP models

With respect to DCA, we consider four different types of model, each varying in the definition of the grammar-type part of the PQC. These models are: DCA-Type IQP, DCA-Type 1, DCA-Type 2, DCA-Type 3. The IQP (Fig. 5) circuit-based model based on the pre-existing functor employed in DisCoCat (Lorenz et al. 2023) is used as a baseline to compare against all these models. Since the baseline model applies the IQP circuit for every grammar type, for simplicity, we refer to it as IQP in the figures drawn in Sect. 4.1. The DCA-Type IQP, DCA-Type 1, DCA-Type 2 models apply the IQP (Fig. 5), Type 1 (Fig. 6), Type 2 (Fig. 7) circuits, respectively for every grammar type.

The Algorithm 2 provides the steps of the protocol used to create a DCA-Type 1 model circuit. The Type-2 circuit created using the Algorithm 3 is an extension of the protocol followed by Type-1. In the compound type, the adjointness of the atomic type is varying depending on the grammar derivation. For example, consider compound types $n.r\ s.r.r\ n.l\ n$ and $n.r\ s.r\ n.l\ n$. Here, the atomic type $s$ has $r$ adjoint twice in the first compound type, while it has $r$ adjoint once in the second. A Type-1 circuit ansatz would not distinguish between these two and will assign the same number of parameters to both types. However, the application of the double adjoint has a definite purpose in grammar construction of that type. So, taking this into account, we count the number of times such adjoint occurs for each atomic type in the compound type and accordingly assign those many rotation gates (Fig. 7), hence, distinguishing between the number and structure of quantum gates associated with both types. The supporting intuition of using such protocols is that adding some structure to the parameterization of grammar types may improve performance and provide better control over the learning routine of each type. This is different from the IQP case where only controlled rotation gates are used to represent grammar types.

The PQCs formed using the Type-IQP, Type-1 and Type-2 ansatz have a single layer of parameterized gates; however, more such layers can be added ahead of the first layer to enhance the expressibility of the circuit. The Type-3 circuit ansatz is similar to the Type-2 ansatz in its application of the protocol to apply the rotation gates. However, it takes into consideration another hyperparameter which depends on the chosen dataset for the NLP task.

With little observation, it is natural to see that certain types will appear more frequently in the dataset than others (refer to Tables 2, 3, 4) and hence the parameters corresponding to them may require more fine-tuning. Thus, such types may need a different level of parameterization than the relatively less frequent types. We execute this using a simple scheme that customarily assigns the number of layers to each type. Let $S$ be the set of all different grammar types in a training corpus and a dictionary $D$ where the grammar types of $S$ are keys with their corresponding value denoting the count of different words in the training vocabulary having that type. Consider $q$ is the qubit count for any given type, $k$ is a predefined percentage factor acting as the hyperparameter, and $lc$ is the predefined layer count similar to the Type-1 and Type-2 ansatzes. Then, the number of circuit layers $L$ in the Type-3 ansatz for a particular type can be determined by the following equations:

$$l = D[type] * k \tag{1}$$

$$L = \begin{cases} l/q, & L > q \\ lc, & L \leq q \end{cases} \tag{2}$$

In Eq. 1, the count of unique words for the given type retrieved from $D$ is multiplied by $k$. Later, it is checked if this fraction of a count is at least equal to the qubit count for the type, since more qubits would require more layers to be highly expressible. Depending on this comparison, as given in Eq. 2, the final layer count is set for the parameterized

gates corresponding to the type. Thus, the higher the frequency of the grammar type, the deeper the corresponding quantum circuit. An important thing to note here is that the types are collected only from the training set diagrams. This is to prevent any kind of inherent information leakage into the train set circuits from the evaluation datasets. The predefined percentage factor is an added hyperparameter that can be tuned to improve performance.

**Algorithm 2** Parameterization Protocol for Type-1 ansatz

$Rz$, $Rx$, $Ry$ are parameterized rotation gates, $z$ = level of adjointness, $\tau$ = box type
**if** $\tau$ is atomic **then**
  Apply a $Rz(\theta)$ gate followed by a $Rx(\phi)$ gate
**else**
  **for** each atomic type a in $\tau$ **do**
    **if** $a.z == 0$ **then**
      Apply a $Ry(\theta)$ gate
    **else if** $a.z >= 1$ **then**
      Apply a $Rz(\theta)$ gate followed by a $Ry(\phi)$ gate
    **else**
      Apply a $Ry(\theta)$ gate followed by a $Rz(\phi)$ gate
    **end if**
  **end for**
**end if**

**Algorithm 3** Parameterization Protocol for Type-2 ansatz

$Rz$, $Rx$, $Ry$ are parameterized rotation gates, $z$ = level of adjointness, $\tau$ = box type
**if** $\tau$ is atomic **then**
  Apply a $Rz(\theta)$ gate followed by a $Rx(\phi)$ gate
**else**
  **for** each atomic type a in $\tau$ **do**
    **if** $a.z == 0$ **then**
      Apply a $Ry(\theta)$ gate
    **else if** $a.z >= 1$ **then**
      $zk \leftarrow z$
      **while** $zk! = 0$ **do**
        Apply a $Rz(\theta)$ gate
        Decrement $zk$ by 1
      **end while**
      Apply a $Ry(\phi)$ gate
    **else**
      Apply a $Ry(\phi)$ gate
      $zk \leftarrow z$
      **while** $zk! = 0$ **do**
        Apply a $Rz(\theta)$ gate
        Increment $zk$ by 1
      **end while**
    **end if**
  **end for**
**end if**

### 3.3 NLP tasks

To validate this novel DCA approach, we test it on two major NLP tasks, namely sentence similarity and paraphrase identification, formalized as a regression and classification task, respectively. One challenging aspect of these tasks is to deal with short text (Kajiwara et al. 2017). If two sentences have many lexical features in common, then it is possible that the semantic features are also the same between them. However, short text does not exhibit extensive lexical features that can help the model learn. Thus, there arises a bottleneck even while using complex neural network models. On the other hand, in our case, we can deal with only short phrases of 7–8 words currently due to computational limitations. Thus, it seems like a fruitful endeavor to test DCA on these tasks for relatively shorter text.

To quantify the semantic similarity between the sentences, we calculate the quantum state fidelity (Jozsa 1994) of the two output states, each from a pair of sentences. The fidelity between two quantum states provides a measure of overlap between those states, akin to the ubiquitous cosine similarity measure between output vectors in classical NLP.

For two pure quantum states given my unit vectors $|\psi\rangle$ and $|\phi\rangle$, the fidelity $F$ between them is given by,

$$F(\psi, \phi) = |\langle\psi|\phi\rangle|^2 \tag{3}$$

For instance, since we simulate quantum circuits, the open wire at the bottom of Fig. 3 gives us the quantum state postselected on the condition that all other qubits are in the $|0\rangle$ state. Since we are postselecting outputs, the resultant quantum state from each circuit is first normalized before calculating the fidelity. Depending on the task, the loss function and evaluation metrics are selected. For the sentence similarity task, the mean squared error (MSE) is considered as a loss function. For the paraphrase identification task, the binary cross entropy (BCE) also termed *log loss* is used to calculate the loss value. To evaluate the performance of the model on the sentence similarity task, we used the Pearson correlation coefficient as a metric.

If the sentence similarity scores are denoted $y_i$ having mean value $\bar{y}$ and fidelity measures $x_i$ between the circuits formed from those sentences with $\bar{x}$ showing mean of those measures, the correlation coefficient $r$ is given by,

$$r = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2}\sqrt{\sum_{i=1}^{n}(y_i - \bar{y})^2}} \tag{4}$$

For paraphrase identification, the prediction labels are assigned depending on the fidelity value. If it is less than or equal to 0.5, the label 0 is assigned while the label 1 is assigned if the fidelity is greater than 0.5. The accuracy of the model is checked for the paraphrase identification task.

We used the Python programming language for the empirical validation of the DCA approach. The NLTK library (Loper and Bird 2002) was used for preprocessing steps such as tokenization and vocabulary creation. The lambeq library (Kartsaklis et al. 2021) was used to setup and run all the experiments. As mentioned in Sect. 3.1, we used the principal components of word embeddings in DCA. The PCA model from Scikit learn library (Pedregosa et al. 2018) was used with the following argument settings—*n_components* = *mle*, *svd_solver* = *auto*, *random_state* = 42, *whiten* = *True*. All the experiments were performed in Google Colab.[3]

### 3.4 Data analysis and preprocessing

We tested all of our models on three different datasets. For the sentence similarity task, we consider two datasets: the SemEval Semantic Textual Similarity (STS) benchmark dataset (Agirre et al. 2012, 2013, 2014, 2015, 2016) and the Sentences Involving Compositional Knowledge (SICK) dataset (Marelli et al. 2014). STS dataset is a collection of smaller datasets used in the SemEval competitions from 2012 to 2016 and consists of image captions and news. SICK consists of sentence pairs having extensive lexical, syntactic and semantic features. Google recently released a paraphrase dataset called Paraphrase Adversaries from Word Scrambling (PAWS) (Zhang et al. 2019). According to its creators, this dataset includes sentences which are not paraphrases but have high lexical overlap, something which most other paraphrase datasets lack.

For all experiments, we consider sentences with word count less than or equal to six for STS and SICK, and sentences with maximum eight words for PAWS. After removing extra artifacts, such as punctuation that do not seem to alter performance, string diagrams are derived for the sentences. Diagrams that do not reduce to type $s$ are filtered out because these sentences cannot be considered grammatically correct (Coecke et al. 2010). After rewriting and eventual formation of quantum circuits for these diagrams, we restrict the circuit size to 7 qubits for SICK and STS, and 10 qubits for PAWS. To model the OOV problem, we follow a two-step procedure to include unknown words in the evaluation datasets. First, we create a vocabulary for the training set and filter out all those sentences that have words with a total vocabulary count of less than five. This ensures that we are not dealing with too many infrequent words. Later, we randomly handpick a few words each from the development set and test set and make sure all the sentences that contain these words are filtered out from the train set but are present in either the development set or test set.

[3] https://colab.research.google.com/.

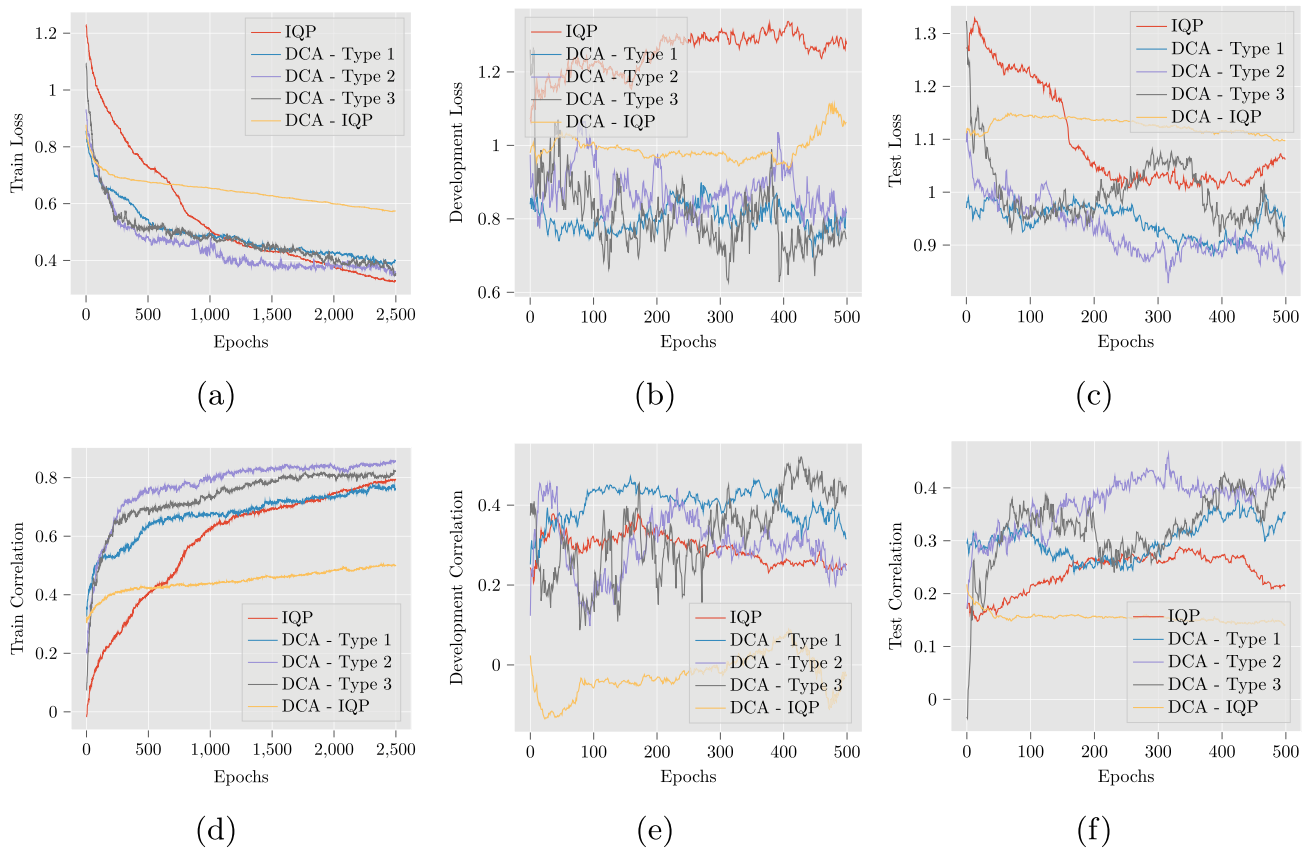



**Fig. 8** Performance of all models on the SICK dataset

## 4 Empirical analysis

This section provides a detailed discussion of the results, including some important limitations.

### 4.1 Results and discussion

Once the model is created following the pipeline outlined in Sect. 2.3, we set up the training routine. We employ 3-layer PQCs for all models except the Type-3 model which has a custom layer count for each grammar type symbol with $k = 0.2$. The training part of all experiments is carried out with the SPSA (Simultaneous Perturbation Stochastic Approximation) optimizer (Spall 1998). The SPSA algorithm approximates the gradient of the loss value with respect to the model parameters instead of analytically calculating it for each parameter. One significant fact about SPSA is that it works especially well in noisy situations. It has proven to be very successful in performing QML tasks (Wiedmann et al. 2023). Collecting useful insights from multiple experiments carried out over the course of the project, we set the final configuration of hyperparameter values for SPSA as follows—$a = 0.0005$, $c = 0.01$, $A = 0.001$, $\alpha = 0.001$, $\gamma = 0.005$. Each model is run for 2500 epochs. The development/test set loss and correlation/accuracy are calculated after every 5 epochs. Since SPSA is a stochastic optimizer, each model is run five times on every dataset. The average of loss values over the runs along with the standard deviation is used to draw the curves displayed in Figs. 8, 9 and 10.

Table 1 lists the counts of sentences and OOV words used for each dataset. It is visible that we deal with very small portions of these datasets. One reason for this is the high computational resource requirements for running DisCoCat-based models. But an even more important fact to note is that we are only considering valid sentences, i.e., those sentences that result in a string diagram having only one open wire of type $s$, as required for the quantum machine learning part of the QNLP pipeline. Surprisingly, as we will see ahead, these smaller counts of sentences actually help validate our intuition that DCA indeed handles a high number of OOV words better than the pre-existing functor used in DisCoCat. Table 2 lists the symbols present in the SICK dataset. The training curves for all five models for SICK are similar to each other and each of them decreases smoothly (Fig. 8a, d). This indicates that the model is successfully learning the train data. For the development set, it is clearly visible

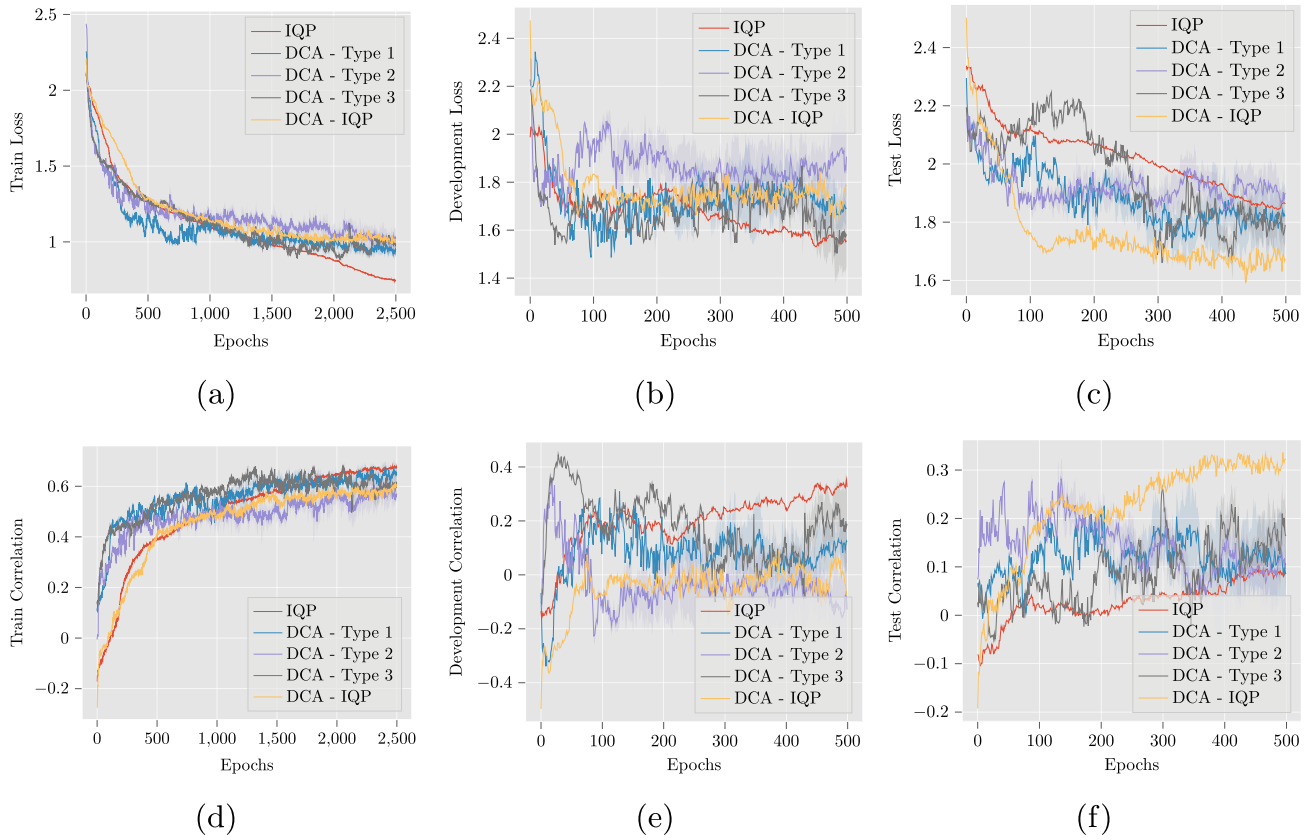



**Fig. 9** Performance of all models on the STS dataset

from Fig. 8b, e that the IQP ansatz-based model fails on this data along with the DCA–IQP model also showing a lack of learning. All other DCA-based models show some promise on the development set. Figure 8c, f depict better performance for Type 1, Type 2 and Type 3 models over the IQP model, while the Type IQP model totally fails on the test dataset.

Table 3 lists the symbols present in the STS dataset. The train loss and correlation curves (Fig. 9a, d) display convergence for all models. Similarly, for the development set, the loss curves seem to decrease where the IQP model performs comparatively better than other models. This can be attributed to the fact that there are very less OOV words in the development set, and a relatively complex circuit hinders performance. Figure 9b, e display better test set performance for Type-1, Type-2 and Type-3 models over the IQP model. Interestingly, the DCA-Type IQP circuit performs best on the STS dataset by a considerable margin over other models. This can be due to the higher count of test set sentences, but with relatively fewer OOV words.

Table 4 lists the symbols present in the PAWS dataset. Figure 10a, d display convergence in the case of the PAWS dataset. The IQP model performs better than other models on the development set as shown by Fig. 10b, e. For the test set (Fig. 10c, f), the Type-3 displays accuracy closer to the IQP and Type-1 circuits; however, the loss and the accuracy curve for Type-3 hint that better performance may be expected if the dataset size is increased. This result clearly displays the power of DCA where the OOV words are high in number with respect to the number of sentences, and the Type-3 circuit with its unique structural formation works better than other circuits.

The results display a distinct trend where the count of total sentences and the OOV words in a dataset affect the performance levels. For datasets with sentence-to-OOV word count ratio less than or close to 1, such as the SICK and PAWS test sets, the DCA-based models, namely Type-2 and Type-3, show better performance than the baseline. Even in the case of the STS test set, the DCA-Type IQP model has higher accuracy even though the OOV word count is relatively lower. This clearly hints that the circuits employed by these models have the necessary structure and amount of parameters that can be fine-tuned depending on the number of OOV words.

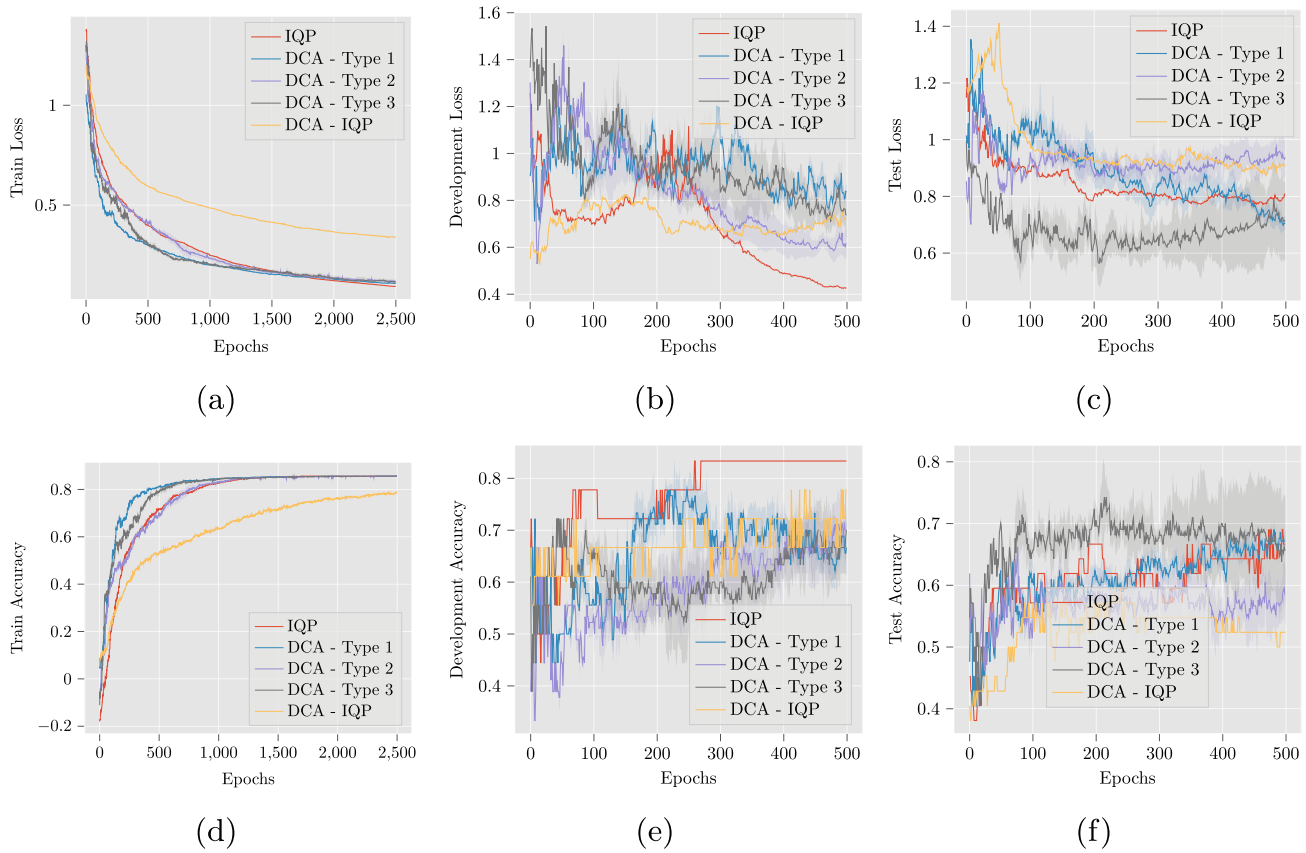



**Fig. 10** Performance of all models on the Google PAWS dataset

**Table 1** Counts of sentences and OOV words in each dataset

| | Train | Development \ OOV words | Test \ OOV words |
|---|---|---|---|
| SICK | 116 | 42\20 | 94\133 |
| STS | 116 | 19\6 | 99\12 |
| PAWS | 106 | 18\15 | 42\47 |

**Table 2** Count of each grammar type symbol for SICK dataset where the value of count denotes the number of different words having that type in train set vocabulary

| Type symbol | count |
|---|---|
| _n | 25 |
| _n.r@s@s.l@n | 3 |
| n.r@s_ | 7 |
| n@n.l_ | 6 |
| _n.r@s@n.l | 1 |
| _n@n.l | 7 |
| n_ | 33 |
| n.r@s@n.l_ | 14 |
| n.r@n@n.l_ | 2 |
| n.r@n_ | 4 |
| _p@n.l | 1 |
| n.r@s@p.l_ | 1 |

**Table 3** Count of each grammar type symbol for STS dataset where the value of count denotes the number of different words having that type in train set vocabulary

| Type symbol | Count |
|---|---|
| _n | 24 |
| _n.r@s@s.l@n | 4 |
| _n@n.l | 4 |
| n_ | 22 |
| n.r@s@n.l_ | 9 |
| n@n.l_ | 7 |
| _n.r@s@n.l | 3 |
| n.r@s_ | 12 |
| _n.r@n@n.l | 1 |
| _s.r@n.r.r@n.r@s@n.l | 1 |

## 4.2 Limitations

The major limitation of the DCA formulation used here is the word representation part. The PCAing of word embedding followed by truncation induces a bottleneck for efficient word representation due to loss of information. Also, the

**Table 4** Count of each grammar type symbol for PAWS dataset where the value of count denotes the number of different words having that type in train set vocabulary

| Type symbol | Count |
|---|---|
| _s@s.l | 2 |
| _n@n.l | 53 |
| _n | 53 |
| n@n.l_ | 55 |
| n.r@s@n.l_ | 8 |
| _s.r@n.r.r@n.r@s@n.l | 4 |
| _n.r@n | 2 |
| n_ | 60 |
| _n.r@s@n.l | 8 |
| _n.r@s@s.l@n | 3 |
| n.r@s_ | 2 |
| _n.r@n@n.l | 1 |
| n.r@n@n.l_ | 2 |
| n.r@n_ | 4 |
| _s@s.l@n.l | 2 |
| _n.r@s@s.l@n@p.l | 1 |
| p@n.l_ | 2 |
| _p@n.l | 3 |
| n.r@s@p.l_ | 3 |
| _n.r@s | 5 |
| s.r@n.r.r@n.r@s@n.l_ | 1 |
| _s@s.l@p.l | 1 |
| _n.r@s@s.l@n@n.l | 1 |
| n.r@s@s.l@n_ | 2 |
| _s.r@n.r.r@n.r@s | 2 |
| s.r@n.r.r@n.r@s_ | 2 |
| s.r@n.r.r@n.r@s@p.l_ | 1 |
| s@s.l@n.l_ | 1 |
| _n.r@s@p.l@p.l | 1 |
| _n.r@s@p.l | 1 |

Mottonen state preparation increases the depth of the circuit, which leads to higher execution times. This depth especially increases rapidly with the number of qubits assigned to the grammar type, eventually resulting in not only a large circuit in terms of qubit count but also a deeper circuit. In the context of quantum machine learning, such deep circuits are known to cause issues like barren plateaus (Ragone et al. 2023). Thus, it is certainly non-trivial to have a good enough quantum state representation considering the complexity of relations between words and the varying number of qubits used by each word. On the other hand, if there are grammar types in the evaluation datasets that are not present in the train set, then the problem once again translates to being an OOV problem, this time due to the grammar types. To ensure that this does not happen, one needs to have a sufficiently large dataset.

On the empirical side, to get the results presented here, we dealt with fewer sentences because of limited computational resources. Hence, the given techniques need to be evaluated on more as well as larger sentences. Even though the DCA approach seems scalable on paper, the implementation of the current formulation instead consumes more resources than the pre-existing technique. This is due to the reasons explained earlier with respect to the word representation part.

## 5 Conclusion and future work

The empirical results serve as evidence that the idea of separating words and their corresponding types indeed handles the OOV problem in a more informed manner. They also suggest that even though it is non-trivial to have an efficient word representation as elaborated in Sect. 4.2, the current formulation seems to model it sufficiently. However, note that it is not a necessity to use only the given technique to capture word information. This subroutine could be replaced by some other way of creating word representations. This novel technique can be constructed in a way such that it supports the quantum circuit formalism more than the one employed in this work. The DCA also ensures scalability due to a huge reduction in the number of model parameters (Table 5). Immediate future work on the topic may involve testing the novel technique on more datasets. In addition, other NLP tasks that involve semantic evaluation of text can be tried to check real-world applicability. Nevertheless, this endeavor to separate the word from the type while constructing the quantum circuit has certainly opened a new direction of research in the DisCoCat-based QNLP domain, a direction that seems worth pursuing considering the quality of the results with relatively less parameter count and the conjecture that it brings forth regarding the word-type separation phenomenon.

**Table 5** Total trainable parameter count for different models on each dataset. 3-layer circuits are used for all models except Type-3 which has custom layer count for each grammar type symbol

| | IQP (baseline) | DCA—Type IQP | DCA—Type 1 | DCA—Type 2 | DCA—Type 3 |
|---|---|---|---|---|---|
| STS | 954 | 60 | 169 | 180 | 172 |
| SICK | 534 | 60 | 175 | 183 | 179 |
| PAWS | 1704 | 198 | 634 | 657 | 677 |

**Data Availability** The data sets generated during and/or analyzed during the current study are available in the open-source. The following are details. 1. Semantic Textual Similarity Wiki: http://ixa2.si.ehu.eus/stswiki/. 2. PAWS: Paraphrase Adversaries from Word Scrambling: https://github.com/google-research-datasets/paws. 3. The SICK data set: Sentences Involving Compositional Knowledge: https://marcobaroni.org/composes/sick.html.

## Declarations

**Conflicts of interest** The authors declare that they have no conflict of interest.

**Ethics approval and consent to participate** This article does not contain any studies with human participants or animals performed by any of the authors.

## References


Agirre E, Banea C, Cardie C et al (2014) SemEval-2014 task 10: multilingual semantic textual similarity. In: Proceedings of the 8th international workshop on semantic evaluation (SemEval 2014). Association for Computational Linguistics, Dublin, Ireland, pp 81–91. https://doi.org/10.3115/v1/S14-2010, Accessed 1 July 2025.

Agirre E, Banea C, Cardie C et al (2015) SemEval-2015 task 2: semantic textual similarity, English, Spanish and Pilot on interpretability. In: Proceedings of the 9th international workshop on semantic evaluation (SemEval 2015). Association for Computational Linguistics, Denver, Colorado, pp 252–263. https://doi.org/10.18653/v1/S15-2045, Accessed 1 July 2025.

Agirre E, Banea C, Cer D et al (2016) SemEval-2016 task 1: semantic textual similarity, monolingual and cross-lingual evaluation. In: Proceedings of the 10th international workshop on semantic evaluation (SemEval-2016). Association for Computational Linguistics, San Diego, California, pp 497–511. https://doi.org/10.18653/v1/S16-1081,

Agirre E, Cer D, Diab M et al (2012) SemEval-2012 task 6: a pilot on semantic textual similarity. In: *SEM 2012: The first joint conference on lexical and computational semantics—volume 1: Proceedings of the main conference and the shared task, and Volume 2: Proceedings of the sixth international workshop on semantic evaluation (SemEval 2012). Association for Computational Linguistics, Montréal, Canada, pp 385–393. https://aclanthology.org/S12-1051

Agirre E, Cer D, Diab M et al (2013) *SEM 2013 shared task: Semantic textual similarity. In: Diab M, Baldwin T, Baroni M (eds) Second joint conference on lexical and computational semantics (*SEM), volume 1: Proceedings of the main conference and the shared task: semantic textual similarity. Association for Computational Linguistics, Atlanta, Georgia, USA, pp 32–43. https://aclanthology.org/S13-1004

Bhat HA, Khanday FA, Kaushik BK et al (2022) Quantum computing: fundamentals, implementations and applications. IEEE Open J Nanotechnol 3:61–77

Bojanowski P, Grave E, Joulin A et al (2017) Enriching word vectors with subword information. arXiv:1607.04606

Coecke B, Sadrzadeh M, Clark S (2010) Mathematical foundations for a compositional distributional model of meaning. arXiv:1003.4394

Dong L, Yang N, Wang W et al (2019) Unified language model pre-training for natural language understanding and generation. In: Wallach H, Larochelle H, Beygelzimer A et al (eds) Advances in neural information processing systems, vol 32. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2019/file/c20bb2d9a50d5ac1f713f8b34d9aac5a-Paper.pdf Accessed 1 July 2025.

Garneau N, Leboeuf JS, Lamontagne L (2018) Predicting and interpreting embeddings for out of vocabulary words in downstream tasks. In: Linzen T, Chrupała G, Alishahi A (eds) Proceedings of the 2018 EMNLP workshop BlackboxNLP: analyzing and interpreting neural networks for NLP. Association for Computational Linguistics, Brussels, Belgium, pp 331–333. https://doi.org/10.18653/v1/W18-5439, https://aclanthology.org/W18-5439

Grave E, Bojanowski P, Gupta P et al (2018) Learning word vectors for 157 languages. In: Proceedings of the international conference on language resources and evaluation (LREC 2018)

Guarasci R, De Pietro G, Esposito M (2022) Quantum natural language processing: challenges and opportunities. Appl Sci 12(11). https://www.mdpi.com/2076-3417/12/11/5651 Accessed 1 July 2025.

Hadi MU, Al-Tashi Q, Qureshi R et al (2023) Large language models: a comprehensive survey of its applications, challenges, limitations, and future prospects. https://doi.org/10.36227/techrxiv.23589741.v1

Harris ZS (1954) Distributional structure. https://api.semanticscholar.org/CorpusID:86680084 Accessed 1 July 2025.

Hawashin H, Sadrzadeh M (2024) Multimodal structure-aware quantum data processing. arXiv preprint arXiv:2411.04242

Huang H, Broughton MB, Cotler J et al (2021) Quantum advantage in learning from experiments. Science 376:1182–1186. https://doi.org/10.1126/science.abn7293

Huang J, Chen L, Yi X et al (2024) Quantum theory-inspired inter-sentence semantic interaction model for textual adversarial defense. Complex Intel Syst. https://doi.org/10.1007/s40747-024-01733-4

Jozsa R (1994) Fidelity for mixed quantum states. J Mod Opt 41(12):2315–2323. https://doi.org/10.1080/09500349414552171

Kajiwara T, Bollegala D, Yoshida Y et al (2017) An iterative approach for the global estimation of sentence similarity. PLoS One 12. https://api.semanticscholar.org/CorpusID:21484145 Accessed 1 July 2025.

Kartsaklis D, Fan I, Yeung R et al (2021) lambeq: An efficient high-level python library for quantum nlp. arXiv:2110.04236

Lambek J (1997) Type grammar revisited. In: Logical aspects of computational linguistics. https://api.semanticscholar.org/CorpusID:35755316. Accessed 1 July 2025.

Loper E, Bird S (2002) Nltk: the natural language toolkit. arXiv:cs/0205028

Lorenz R, Pearson A, Meichanetzidis K et al (2023) Qnlp in practice: running compositional models of meaning on a quantum computer. J Artif Intell Res 76:1305–1342. https://doi.org/10.1613/jair.1.14329

Maćkiewicz A, Ratajczak W (1993) Principal components analysis (pca). Comput Geosci 19(3):303–342. https://doi.org/10.1016/0098-3004(93)90090-R Accessed 1 July 2025.

Marelli M, Menini S, Baroni M et al (2014) A SICK cure for the evaluation of compositional distributional semantic models. In: Proceedings of the ninth international conference on language resources and evaluation (LREC'14). European Language Resources Association (ELRA), Reykjavik, Iceland, pp 216–223. http://www.lrec-conf.org/proceedings/lrec2014/pdf/363_Paper.pdf

Meichanetzidis K, Gogioso S, de Felice G et al (2021) Quantum natural language processing on near-term quantum computers. Electron Proc Theor Comput Sci 340:213–229. https://doi.org/10.4204/eptcs.340.11

Minka T (2000) Automatic choice of dimensionality for pca. In: Leen T, Dietterich T, Tresp V (eds) Advances in neural information

processing systems, vol 13. MIT Press. https://proceedings.neurips.cc/paper_files/paper/2000/file/7503cfacd12053d309b6bed5c89de212-Paper.pdf. Accessed 1 July 2025.

Möttönen M, Vartiainen J, Bergholm V et al (2005) Transformation of quantum states using uniformly controlled rotations. Quantum Inf Comput 5:467–473. https://doi.org/10.26421/QIC5.6-5

Pedregosa F, Varoquaux G, Gramfort A et al (2018) Scikit-learn: machine learning in python. arXiv:1201.0490

Pinter Y, Guthrie R, Eisenstein J (2017) Mimicking word embeddings using subword RNNs. In: Palmer M, Hwa R, Riedel S (eds) Proceedings of the 2017 conference on empirical methods in natural language processing. Association for Computational Linguistics, Copenhagen, Denmark, pp 102–112. https://doi.org/10.18653/v1/D17-1010

Ragone M, Bakalov BN, Sauvage F et al (2023) A unified theory of barren plateaus for deep parametrized quantum circuits. arXiv:2309.09342

Rubio-Perez JP (2024) A few shot learning scheme for quantum natural language processing. arXiv:2410.01832

Sasaki S, Suzuki J, Inui K (2019) Subword-based compact reconstruction of word embeddings. In: Burstein J, Doran C, Solorio T (eds) Proceedings of the 2019 conference of the North American chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers). Association for Computational Linguistics, Minneapolis, Minnesota, pp 3498–3508. https://doi.org/10.18653/v1/N19-1353,

Shepherd D, Bremner MJ (2009) Temporally unstructured quantum computation. Proc R Soc A Math Phys Eng Sci 465(2105):1413–1439. https://doi.org/10.1098/rspa.2008.0443

Shor PW (1997) Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. SIAM J Comput 26(5):1484–1509. https://doi.org/10.1137/s0097539795293172

Smaldone AM, Shee Y, Kyro GW et al (2025) A hybrid transformer architecture with a quantized self-attention mechanism applied to molecular generation. arXiv:2502.19214

Spall JC (1998) Implementation of the simultaneous perturbation algorithm for stochastic optimization. IEEE Trans Aerosp Electron Syst 34(3):817–823. https://doi.org/10.1109/7.705889

Steedman M, Baldridge J (2011) Non-transformational syntax: formal and explicit models of grammar, pp 181–224. https://doi.org/10.1002/9781444395037.ch5

Tomal S, Shafin AA, Bhattacharjee D et al (2025) Quantum-enhanced attention mechanism in nlp: a hybrid classical-quantum approach. arXiv preprint arXiv:2501.15630

Wang D, Sadrzadeh M, Abramsky S et al (2021) On the quantum-like contextuality of ambiguous phrases. arXiv:2107.14589

Wazni H, Lo KI, McPheat L et al (2024) Large scale structure-aware pronoun resolution using quantum natural language processing. Quantum Mach Intell 6(2):60

Wiedmann M, Höölle M, Periyasamy M et al (2023) An empirical comparison of optimizers for quantum machine learning with spsa-based gradients. In: 2023 IEEE international conference on quantum computing and engineering (QCE). IEEE. https://doi.org/10.1109/qce57702.2023.00058

Wu S, Li J, Zhang P et al (2021) Natural language processing meets quantum physics: a survey and categorization. In: Moens MF, Huang X, Specia L et al (eds) Proceedings of the 2021 conference on empirical methods in natural language processing. Association for Computational Linguistics, Online and Punta Cana, Dominican Republic, pp 3172–3182. https://doi.org/10.18653/v1/2021.emnlp-main.254

Zhang Y, Baldridge J, He L (2019) Paws: paraphrase adversaries from word scrambling. arXiv:1904.01130

Zhu J, Ma X, Lin Z et al (2023) A quantum-like approach for text generation from knowledge graphs. CAAI Trans Intell Technol 8(4):1455–1463. https://doi.org/10.1049/cit2.12178

**Publisher's Note** Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.